

COMPTE RENDU

Ethical Hacking & Defense - TP1 - Footprinting et Reconnaissance
4ème année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants : HATHOUTI Mohammed Taha
JIDAL Ilyas
Filière : Cybersecurité
Année : 2026/2027
Enseignant : M.SEBBAR & M.ZEROUAL
Date : 22 septembre 2026

Table des matières

Objectifs	3
Précautions Légales et Éthiques	3
Environnement de Travail	3
1 Partie 1 : Footprinting via les outils de diagnostic réseau	4
1.1 Recherche de l'adresse IP du domaine	4
1.1.1 Principe	4
1.1.2 Exécution	4
1.1.3 Analyse des résultats	4
1.2 Détermination de la taille de trame maximale (MTU)	5
1.2.1 Objectif et rappel théorique	5
1.2.2 Adaptation de la commande sous Linux	5
1.2.3 Recherche du seuil par dichotomie	6
1.2.4 Synthèse et interprétation	7
1.2.5 Intérêt de cette mesure en reconnaissance	7
1.3 Étude du TTL (<i>Time To Live</i>)	8
1.3.1 Principe	8
1.3.2 Adaptation de la commande sous Linux	8
1.3.3 Exécution avec la commande Linux correcte	9
1.3.4 Analyse des résultats	9
1.4 Traçage de la route vers le domaine	10
1.4.1 Principe	10
1.4.2 Exécution	10
1.4.3 Analyse saut par saut	11
1.4.4 Lien avec les remarques de l'énoncé	11
1.5 Vérification de la durée de vie du paquet	12
1.5.1 Principe	12
1.5.2 Adaptation de la commande sous Linux	12
1.5.3 Exécution	13
1.5.4 Analyse des résultats	14
1.6 Informations DNS avec <code>nslookup</code>	15
1.6.1 Principe	15
1.6.2 Requête de type A	15
1.6.3 Requêtes de type CNAME puis NS	15
1.6.4 Intérêt en reconnaissance	17
1.7 Adresse IP du serveur de noms faisant autorité	17
1.7.1 Principe	17
1.7.2 Requête de type A sur <code>ns1.bluehost.com</code>	18
1.7.3 Interrogation directe du serveur faisant autorité	18
1.7.4 Intérêt en reconnaissance	19

1.8	Footprinting via les moteurs de recherche (<i>Google Dorking</i>)	19
1.8.1	Principe	19
1.8.2	Recherche de documents PDF	20
1.8.3	Recherche de portails de connexion	20
1.8.4	Recherche d'adresses e-mail	21
1.8.5	Limites de la méthode	22
1.8.6	Synthèse des trois requêtes	23
1.9	Footprinting via theHarvester	23
1.9.1	Principe	23
1.9.2	Écart avec l'énoncé	23
1.9.3	Recherche via DuckDuckGo	23
1.9.4	Recherche via crt.sh (transparence des certificats)	24
1.9.5	Intérêt en reconnaissance	25
1.10	Analyse de dépôts OSINT avancés (Shodan et pagodo)	26
1.10.1	Principe	26
1.10.2	Réponse à la question de l'énoncé (Shodan)	26
2	Partie 2 : Footprinting via WHOIS, DNS, reconnaissance réseau et analyse de site web	27
2.1	WHOIS Footprinting	27
2.1.1	Principe	27
2.1.2	Analyse des résultats	28
2.2	DNS Footprinting	29
2.2.1	Principe	29
2.2.2	Énumération avec dig	29
2.2.3	Serveurs de noms avec nslookup	30
2.3	Reconnaissance réseau (Nmap)	31
2.3.1	Principe et précaution	31
2.3.2	Détection des services et versions	31
2.3.3	Détection du système d'exploitation	33
2.4	Analyse du site web (wget & Nikto)	34
2.4.1	Miroir du site avec wget	34
2.4.2	Scan de vulnérabilités avec Nikto	35
2.5	Réponses aux questions de l'énoncé	36
2.6	Exercices théoriques (Task 2)	36
3	Partie 3 : Automatisation du footprinting (script shell)	37
3.1	Objectif	37
3.2	Script	37
3.3	Exécution et observations	38
3.4	Réponses aux questions de l'énoncé	40

Objectifs

L'objectif de ce TP est d'acquérir une expérience pratique des techniques de *footprinting* et de reconnaissance, afin de comprendre comment un attaquant collecte des informations sur une cible avant de lancer une attaque. La reconnaissance constitue la première phase de toute démarche offensive : plus la collecte passive d'informations est riche, plus la surface d'attaque devient exploitable.

Ce TP est organisé en trois parties :

- **Partie 1** : Footprinting via les moteurs de recherche, les services web et les réseaux sociaux ;
- **Partie 2** : Footprinting via WHOIS, DNS, reconnaissance réseau et analyse de site web ;
- **Partie 3** : Utilisation d'outils spécialisés et automatisation du footprinting.

Précautions Légales et Éthiques

IMPORTANT : Les techniques présentées permettent de collecter des informations pouvant servir à des actes illégaux. Elles ne doivent jamais être appliquées à des systèmes réels sans autorisation écrite. Le domaine `certifiedhacker.com` utilisé dans ce TP est une cible d'entraînement publique et légale. La Partie 1 relève de la reconnaissance passive ; la Partie 2 introduit des actions semi-actives (scan de ports, analyse du serveur web), également menées uniquement sur cette cible d'entraînement autorisée.

Environnement de Travail

- **Machine 1 (Partie 1)** : `hathouti@Taha-inspiron-16` — Ubuntu 24.04 ;
- **Machine 2 (Partie 2)** : `ygghest@ygghest-ExMach-Lap` — Arch Linux (poste du binôme) ;
- **Connexion** : accès Internet domestique par fibre optique, encapsulation **PPPoE** (ce point aura son importance en Partie 1, tâche 2) ;
- **Cible** : `www.certifiedhacker.com`.

Répartition du travail : la Partie 1 a été réalisée sur la machine 1 (Ubuntu) et la Partie 2 sur la machine 2 (Arch Linux). Les deux postes partagent le **même réseau local et la même cible**, ce qui explique la forte cohérence des résultats d'une partie à l'autre (même IP cible `162.241.216.11`, même distance de **12 sauts**, même hébergeur **Bluehost**) ainsi que quelques différences d'environnement visibles sur les captures (résolveur DNS, invite du terminal).

Note d'adaptation : L'énoncé du TP est rédigé pour un environnement **Windows** (Invite de commandes / PowerShell). Nos postes étant sous **Linux**, nous avons adapté chaque commande à son équivalent Linux et documenté systématiquement les différences de syntaxe et de comportement, ce qui constitue en soi un apport pédagogique.

1 Partie 1 : Footprinting via les outils de diagnostic réseau

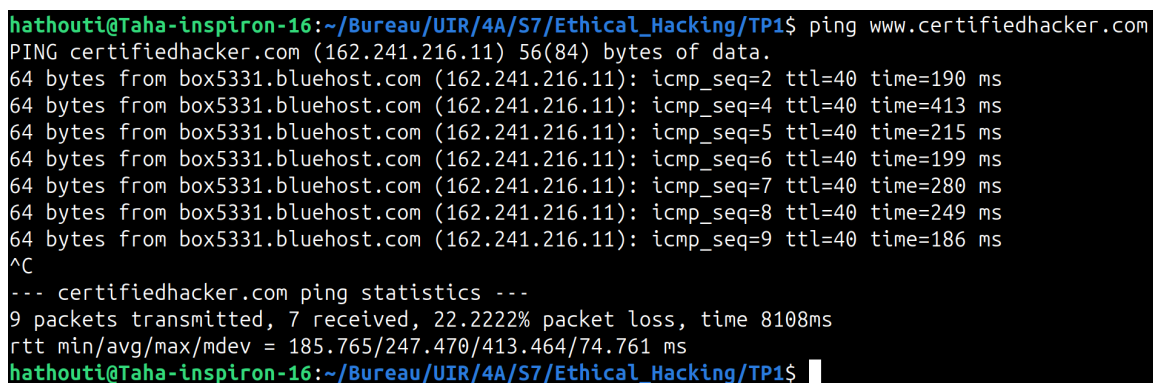
1.1 Recherche de l'adresse IP du domaine

1.1.1 Principe

La commande `ping` envoie des paquets **ICMP Echo Request** vers la cible et mesure le temps d'aller-retour (*RTT*) des **ICMP Echo Reply**. Au-delà de la mesure de latence, elle réalise implicitement une **résolution DNS** qui révèle l'adresse IP de la cible : c'est la première brique de tout footprinting.

1.1.2 Exécution

```
1 ping www.certifiedhacker.com
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data:
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=2 ttl=40 time=190 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=4 ttl=40 time=413 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=5 ttl=40 time=215 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=6 ttl=40 time=199 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=7 ttl=40 time=280 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=8 ttl=40 time=249 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=9 ttl=40 time=186 ms
^C
--- certifiedhacker.com ping statistics ---
9 packets transmitted, 7 received, 22.2222% packet loss, time 8108ms
rtt min/avg/max/mdev = 185.765/247.470/413.464/74.761 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$
```

FIGURE 1 – Résolution de l'adresse IP et mesure de latence vers `www.certifiedhacker.com`

Différence Windows / Linux : sous Windows, `ping` envoie 4 requêtes puis s'arrête automatiquement. Sous Linux, il envoie des requêtes en continu jusqu'à interruption manuelle (`Ctrl+C`), d'où l'usage de `-c` pour fixer un nombre de paquets dans les tâches suivantes.

1.1.3 Analyse des résultats

- **Adresse IP :** 162.241.216.11 ;
- **Nom d'hôte inverse :** box5331.bluehost.com. Cette information est précieuse en reconnaissance : elle révèle que la cible est hébergée sur une **infrastructure mutualisée de Bluehost** (*shared hosting*). Un attaquant en déduit que d'autres sites peuvent partager la même IP et que l'infrastructure n'est pas auto-hébergée ;
- **TTL reçu :** 40. En partant d'une valeur initiale standard de 64 (typique des systèmes Linux/Unix), on estime le nombre de sauts à environ $64 - 40 = 24$ hops, ce qui suggère un serveur distant sous un OS de type Unix ;
- **Latence (RTT) :** variable entre 186 ms et 413 ms, avec une moyenne de 247 ms et une gigue (*mdev*) élevée de 74 ms, cohérente avec un serveur géographiquement distant.

Réponses aux questions de l'énoncé :

- Quelle est l'adresse IP de *www.certifiedhacker.com* ?
→ 162.241.216.11.
- Combien de temps a mis le paquet pour voyager ?
→ De 186 à 413 ms selon le paquet, soit une moyenne de 247 ms.

1.2 Détermination de la taille de trame maximale (MTU)

1.2.1 Objectif et rappel théorique

Le **MTU** (*Maximum Transmission Unit*) est la taille maximale, en octets, d'un paquet que le réseau accepte de transporter sans le fragmenter. L'objectif de cette tâche est de le déterminer expérimentalement en envoyant des paquets de taille croissante avec interdiction de fragmentation, jusqu'à trouver le seuil de rejet.

Il faut distinguer la **charge utile** (les données que l'on envoie) du **paquet complet** qui circule sur le réseau. Le noyau ajoute automatiquement deux en-têtes :

Élément	Taille
En-tête IP (IPv4, sans options)	20 octets
En-tête ICMP (Echo Request)	8 octets
Surcoût total	28 octets

TABLE 1 – Décomposition des 28 octets d'en-têtes ajoutés à chaque ping

On a donc la relation fondamentale :

$$\text{Paquet complet} = \text{charge utile} (-s) + 28 \quad \text{et} \quad \text{MTU} = \text{charge utile max} + 28$$

1.2.2 Adaptation de la commande sous Linux

L'énoncé propose la commande Windows `ping -f -l <taille>`. Sous Linux, ces options ont une signification **totale**ment **différente** et provoquent une erreur :

Option	Signification Windows	Signification Linux (iputils)
-f	Ne pas fragmenter (bit <i>Don't Fragment</i>)	<i>Flood ping</i> (envoi en rafale)
-l	Taille de la charge utile	<i>Preload</i> (plafonné à 3 sans <code>root</code>)

TABLE 2 – Divergence de sémantique des options -f et -l entre Windows et Linux

L'équivalent Linux correct est donc :

```
1 ping -c 7 www.certifiedhacker.com -M do -s <taille>
```

où `-M do` active le bit *Don't Fragment* (véritable équivalent du `-f` de Windows) et `-s` fixe la charge utile (équivalent du `-l`).

1.2.3 Recherche du seuil par dichotomie

Nous avons envoyé des paquets de taille décroissante jusqu'à obtenir des réponses.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 7 www.certifiedhacker.com -M do -s 1500
PING www.certifiedhacker.com (162.241.216.11) 1500(1528) bytes of data.
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
--- www.certifiedhacker.com ping statistics ---
7 packets transmitted, 0 received, +7 errors, 100% packet loss, time 6128ms
```

FIGURE 2 – -s 1500 (paquet de 1528 o) : échec, MTU signalé = 1500

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 7 www.certifiedhacker.com -M do -s 1492
PING www.certifiedhacker.com (162.241.216.11) 1492(1520) bytes of data.
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
--- www.certifiedhacker.com ping statistics ---
7 packets transmitted, 0 received, +7 errors, 100% packet loss, time 6155ms
```

FIGURE 3 – -s 1492 (paquet de 1520 o) : échec, MTU signalé = 1500

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 7 www.certifiedhacker.com -M do -s 1472
PING www.certifiedhacker.com (162.241.216.11) 1472(1500) bytes of data.
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
ping: local error: message too long, mtu=1492
--- www.certifiedhacker.com ping statistics ---
7 packets transmitted, 0 received, +7 errors, 100% packet loss, time 6174ms
```

FIGURE 4 – -s 1472 (paquet de 1500 o) : échec, mais MTU signalé = 1492

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 7 www.certifiedhacker.com -M do -s 1464
PING www.certifiedhacker.com (162.241.216.11) 1464(1492) bytes of data.
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=1 ttl=40 time=239 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=2 ttl=40 time=226 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=3 ttl=40 time=215 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=4 ttl=40 time=235 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=5 ttl=40 time=188 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=6 ttl=40 time=185 ms
1472 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=7 ttl=40 time=181 ms
--- www.certifiedhacker.com ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6008ms
rtt min/avg/max/mdev = 180.880/209.846/238.943/22.994 ms
```

FIGURE 5 – -s 1464 (paquet de 1492 o) : succès, 0% de perte

1.2.4 Synthèse et interprétation

Charge utile (-s)	Paquet total	Résultat	MTU signalé
1500	1528	Échec	1500
1492	1520	Échec	1500
1472	1500	Échec	1492
1464	1492	Succès (0% perte)	—

TABLE 3 – Résultats de la recherche du MTU par tailles décroissantes

Observation cruciale — deux MTU distincts sont révélés : les captures ne signalent pas toutes la même valeur d’erreur, et ce n’est pas un hasard. Elles mettent en évidence **deux goulots d’étranglement successifs** sur le trajet :

1. **Le MTU de l’interface locale (1500)** — Pour `-s 1500` et `-s 1492`, le paquet complet (1528 et 1520 o) dépasse déjà le MTU de la carte réseau Ethernet locale (1500 o). Le noyau rejette donc le paquet **immédiatement, en local**, sans jamais l’émettre, en signalant `mtu=1500`.
2. **Le MTU du chemin / Path MTU (1492)** — Pour `-s 1472`, le paquet complet vaut exactement 1500 o : il **pass**e le contrôle de l’interface locale et part sur le réseau. Il se heurte alors au lien **PPPoE** en amont (celui du fournisseur d’accès), dont le MTU n’est que de 1492 o. Le mécanisme de *Path MTU Discovery* renvoie une erreur ICMP « fragmentation needed », et le noyau signale désormais `mtu=1492`.

Pourquoi le MTU vaut-il 1492 et non 1500 ? La valeur 1492 est la signature classique de l’encapsulation **PPPoE** (*PPP over Ethernet*), utilisée par de nombreux fournisseurs d’accès. Le protocole PPP, qui gère l’authentification et la session de l’abonné, ajoute son propre en-tête de 8 octets par-dessus l’Ethernet :

$$1500 \text{ (Ethernet)} - 8 \text{ (en-tête PPPoE)} = 1492 \text{ (MTU effectif)}$$

Ce résultat n’est donc **pas lié à la fibre** en elle-même (la fibre n’est qu’un support physique qui transporte de l’Ethernet), mais bien à un **choix d’encapsulation de l’opérateur**.

Confirmation que 1464 est bien le maximum. La dichotomie encadre le seuil entre `-s 1464` (succès) et les valeurs supérieures (échec), mais 1464 est aussi **démontrée comme maximale par le calcul** : le noyau signale un MTU de 1492 o, or une charge utile de 1464 o produit un paquet de $1464 + 28 = 1492$ o, soit exactement le MTU. Toute charge utile ≥ 1465 o donnerait un paquet ≥ 1493 o, qui dépasse le MTU et échoue. La commande `ping -c 4 www.certifiedhacker.com -M do -s 1465` le confirmerait expérimentalement (échec attendu). 1464 o est donc la charge utile maximale, et 1492 o la taille de paquet maximale.

1.2.5 Intérêt de cette mesure en reconnaissance

La détermination du MTU dépasse le simple exercice réseau : c’est une information de reconnaissance à part entière.

- Un MTU de **1492** trahit la présence de **PPPoE**, donc un accès de type grand public (DSL/fibre opérateur) ;

- Un MTU inférieur (~1420) trahirait la présence d'un **tunnel VPN** (WireGuard, IPsec) sur le chemin ;
- Connaître la taille utile maximale (1464 o) est indispensable pour dimensionner un éventuel **canal caché ICMP** (*ICMP tunneling*), technique d'exfiltration de données déguisées en pings ;
- La maîtrise du MTU et de la fragmentation est la base des techniques d'**évasion d'IDS/pare-feu** (ex. option `-f` de `nmap`, abordée en Partie 2).

Précision terminologique : « trame » ou MTU ? L'énoncé parle de « taille de trame maximale », mais la valeur mesurée par `ping` est le **MTU**, qui borne le **paquet IP** (couche 3), soit 1492 o. La **trame Ethernet** correspondante (couche 2) englobe en plus l'en-tête Ethernet (14 o) et le champ de contrôle FCS (4 o), soit 18 o supplémentaires. Par convention dans cet exercice, « taille de trame maximale » désigne le MTU ; nous répondons donc 1492 o, en gardant à l'esprit cette distinction couche 2 / couche 3.

Réponse à la question de l'énoncé :

- *Quelle est la taille de trame maximale supportée par votre réseau ?*
→ Le **MTU est de 1492 octets**, correspondant à une charge utile ICMP maximale de **1464 octets** ($1464 + 28 = 1492$). Cette réduction par rapport aux 1500 octets de l'Ethernet standard est due à l'encapsulation **PPPoE** du fournisseur d'accès.

1.3 Étude du TTL (*Time To Live*)

1.3.1 Principe

Le **TTL** est un champ de 8 bits de l'en-tête IP. L'émetteur l'initialise (typiquement 64 sous Linux, 128 sous Windows, 255 sur les équipements Cisco) et **chaque routeur traversé le décrémente de 1**. Lorsqu'un routeur reçoit un paquet dont le TTL arrive à 0, il le **détruit** et renvoie à l'émetteur un message **ICMP Time Exceeded** (type 11, code 0) qui contient sa propre adresse IP.

En fixant volontairement un TTL faible, on force donc le routeur situé à cette distance à se manifester : c'est le mécanisme exploité par `traceroute` (Tâche 4).

1.3.2 Adaptation de la commande sous Linux

L'énoncé propose `ping www.certifiedhacker.com -i 3`. Comme pour `-f` et `-l` à la Tâche 2, cette option a une signification différente sous Linux :

Option	Signification Windows	Signification Linux (iputils)
<code>-i</code>	Valeur du TTL	Intervalle entre deux paquets (en secondes)
<code>-t</code>	Timestamp (<i>tracer</i>)	Valeur du TTL

TABLE 4 – Divergence de sémantique des options `-i` et `-t` entre Windows et Linux

Preuve de la divergence. L'exécution de la commande de l'énoncé telle quelle ne produit pas l'erreur attendue : les réponses arrivent normalement avec `ttl=40` et 0% de perte. Le temps total (6005 ms pour 3 paquets) confirme que `-i 3` a simplement espacé les envois de 3 secondes.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 3 www.certifiedhacker.com -i 3
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=1 ttl=40 time=194 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=2 ttl=40 time=199 ms
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=3 ttl=40 time=198 ms

--- certifiedhacker.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 6005ms
rtt min/avg/max/mdev = 194.490/197.186/198.962/1.938 ms
```

FIGURE 6 – `ping -i 3` sous Linux : intervalle de 3 s entre paquets, et non TTL (aucune expiration)

1.3.3 Exécution avec la commande Linux correcte

```
1 ping -c 3 -t 3 www.certifiedhacker.com
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping -c 3 -t 3 www.certifiedhacker.com
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 10.43.12.209 icmp_seq=1 Time to live exceeded
From 10.43.12.209 icmp_seq=2 Time to live exceeded
From 10.43.12.209 icmp_seq=3 Time to live exceeded

--- certifiedhacker.com ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2002ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$
```

FIGURE 7 – `ping -t 3` : le paquet expire au 3^e saut

1.3.4 Analyse des résultats

- **Message reçu** : *Time to live exceeded*, équivalent Linux du *TTL expired in transit* de Windows. Aucune réponse de la cible : 3 paquets émis, 0 reçus, 100 % de perte ;
- **Routeur émetteur** : 10.43.12.209. Avec un TTL de 3, le paquet est détruit par le 3^e routeur du chemin, qui signale son adresse ;
- **Adresse privée** : 10.43.12.209 appartient au bloc RFC 1918 10.0.0.0/8, non routable sur Internet. Le 3^e saut se situe donc dans le **réseau interne de l'opérateur** (ou derrière un NAT). Cela rejoint la remarque de l'énoncé sur les IP privées en début de chemin, utiles pour déduire l'architecture du réseau ; nous le vérifierons avec `traceroute` (Tâche 4) ;
- **Durée** : 2002 ms pour 3 paquets, soit l'intervalle par défaut de 1 s entre deux envois.

Réponses aux questions de l'énoncé :

- *Que représente le TTL en réseau ?*
→ C'est la **durée de vie d'un paquet IP**, exprimée en nombre de sauts (routeurs traversés) et non en secondes. Il est décrémenté de 1 à chaque routeur, et le paquet est détruit lorsqu'il atteint 0.
- *Pourquoi le TTL est-il important dans le routage des paquets ?*
→ Il **empêche les paquets de circuler indéfiniment** en cas de boucle de routage, ce qui saturerait le réseau. Il permet aussi de **cartographier le chemin** (c'est le principe de `traceroute`) et, par la valeur initiale du TTL reçu, d'**estimer l'OS** de la cible (64 : Unix/Linux, 128 : Windows, 255 : équipements réseau).

1.4 Traçage de la route vers le domaine

1.4.1 Principe

`traceroute` exploite le mécanisme du TTL vu à la Tâche 3. Il envoie des sondes avec un TTL de 1, puis 2, puis 3, etc. Chaque routeur qui détruit une sonde (TTL arrivé à 0) renvoie un message **ICMP Time Exceeded** et révèle ainsi son adresse. En augmentant le TTL de 1 à chaque étape, on reconstitue **saut par saut** le chemin jusqu'à la cible. Trois sondes sont envoyées par saut, d'où les trois temps affichés par ligne.

Différence Windows / Linux : la commande `tracert` de Windows utilise des sondes **ICMP Echo**, alors que `traceroute` sous Linux envoie par défaut des datagrammes **UDP** vers des ports élevés (à partir de 33434). Cette différence a une conséquence pratique : un pare-feu qui filtre l'un des deux protocoles peut produire des résultats différents.

1.4.2 Exécution

```
1 traceroute www.certifiedhacker.com
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ traceroute www.certifiedhacker.com
traceroute to www.certifiedhacker.com (162.241.216.11), 30 hops max, 60 byte packets
 1 fiberbox.home (192.168.11.1)  3.625 ms  3.672 ms  3.785 ms
 2 197.247.192.3 (197.247.192.3)  20.114 ms  20.096 ms  20.080 ms
 3 10.43.12.209 (10.43.12.209)  17.431 ms  19.138 ms  19.121 ms
 4 10.43.12.210 (10.43.12.210)  20.108 ms  20.092 ms  20.062 ms
 5 10.43.250.189 (10.43.250.189)  47.906 ms  49.571 ms  49.556 ms
 6 * * *
 7 * * *
 8 * ae2.3601.edge2.Phoenix1.net.lumen.tech (4.69.219.218)  196.214 ms  196.186 ms
 9 4.18.104.126 (4.18.104.126)  199.838 ms  199.142 ms *
10 140.91.194.165 (140.91.194.165)  197.842 ms  140.91.194.163 (140.91.194.163)  199.101 ms  140.204.226.83 (140.204.226.83)  197.905 ms
11 * * *
12 box5331.bluehost.com (162.241.216.11)  189.951 ms  179.809 ms  186.558 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$
```

FIGURE 8 – Route vers `www.certifiedhacker.com` : 12 sauts

1.4.3 Analyse saut par saut

Saut	Adresse / nom	RTT (ms)	Interprétation
1	fiberbox.home (192.168.11.1)	3,6–3,8	Box domestique, IP privée RFC 1918 : passerelle du réseau local
2	197.247.192.3	≈ 20	Première IP publique : équipement d'accès de l'opérateur (+16 ms, probablement la liaison fibre jusqu'au réseau de l'opérateur)
3	10.43.12.209	17–19	IP privée : cœur de réseau de l'opérateur. Même IP que celle vue à la Tâche 3
4	10.43.12.210	≈ 20	IP contiguë au saut 3 : probablement l'autre extrémité d'un lien point à point
5	10.43.250.189	48–50	Toujours privée ; +30 ms : lien plus long vers un autre site de l'opérateur
6–7	* * *	—	Aucune réponse (ICMP filtré ou limité en débit)
8	ae2.3601.edge2. Phoenix1.net.lumen.tech (4.69.219.218)	≈ 196	Routeur de bord de Lumen à Phoenix (Arizona), d'après son nom. +147 ms : traversée intercontinentale
9	4.18.104.126	≈ 199	Toujours sur le backbone américain
10	140.91.194.165, 140.91.194.163, 140.204.226.83	198–199	Trois IP différentes pour un même saut : équilibrage de charge
11	* * *	—	Aucune réponse (probable filtrage ICMP en amont de la cible)
12	box5331.bluehost.com (162.241.216.11)	180–190	Cible atteinte

TABLE 5 – Interprétation des 12 sauts du traceroute

1.4.4 Lien avec les remarques de l'énoncé

- **Étoiles (* * *) aux sauts 6, 7 et 11** : ces routeurs ne répondent pas aux sondes (ICMP filtré ou limité en débit), mais le trafic les traverse bien, puisque la suite du chemin répond. Un *traceroute* TCP vers le port 80 ou 443 permettrait de contourner ce filtrage ;
- **RTT élevé au saut N , plus bas ensuite** : les sauts 9 et 10 affichent ≈ 199 ms alors que la cible (saut 12) répond en 180–190 ms. Un routeur intermédiaire génère ses réponses ICMP avec une faible priorité, sans que le transit soit réellement plus lent ;
- **Plusieurs IP pour un même saut** : le saut 10 en affiche trois. Le *traceroute* classique change le port de destination à chaque sonde, ce qui modifie le calcul de hachage des routeurs et envoie les sondes sur des chemins parallèles (*load balancing* ECMP). *Paris-traceroute* résout ce problème en gardant les mêmes en-têtes de flux pour toutes les sondes ;
- **IP privées en début de chemin** : les sauts 1, 3, 4 et 5 sont en adresses privées RFC 1918. On en déduit l'architecture : réseau local (192.168.0.0/16), puis réseau interne de l'opérateur en 10.43.x.x, invisible depuis Internet ;
- **Sauts de latence** : trois paliers nets apparaissent (+16 ms au saut 2, +30 ms au saut 5, +147 ms au saut 8). Contrairement à un simple ralentissement ICMP,

le RTT **reste** élevé après le saut 8 : il s'agit d'une vraie distance, la traversée intercontinentale entre le Maroc et les États-Unis.

Réponses aux questions de l'énoncé :

- *Combien de sauts faut-il pour atteindre `www.certifiedhacker.com` ?*
→ **12 sauts**, dont 3 muets (6, 7 et 11).
- *Quelle information chaque saut fournit-il ?*
→ L'**adresse IP** du routeur (et son nom DNS inverse quand il existe), et le **RTT** de trois sondes. Le nom peut révéler l'opérateur, la ville et le type d'interface (ici `Phoenix1.net.lumen.tech`), et le type d'adresse (privée ou publique) renseigne sur la topologie du réseau.

1.5 Vérification de la durée de vie du paquet

1.5.1 Principe

Cette tâche applique manuellement ce que `traceroute` automatise : on envoie un seul paquet avec un TTL croissant (2, 3, 4, ...) jusqu'à ce que la cible réponde elle-même. Tant que le TTL est inférieur au nombre de sauts, un routeur intermédiaire détruit le paquet et signale son adresse ; dès que le TTL est suffisant, le paquet atteint la cible.

1.5.2 Adaptation de la commande sous Linux

L'énoncé propose `ping -i <ttl> -n 1`. Avec les équivalences établies aux tâches précédentes (`-i` → `-t` pour le TTL, `-n` → `-c` pour le nombre de paquets), la commande Linux est :

```
1 ping www.certifiedhacker.com -t <ttl> -c 1
```

1.5.3 Exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 2 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 197.247.192.3 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 3 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 10.43.12.209 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 4 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 10.43.12.210 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 5 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 10.43.250.189 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 6 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 7 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

FIGURE 9 – TTL de 2 à 7 : routeurs intermédiaires, puis deux sauts muets

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 8 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From ae2.3601.edge2.Phoenix1.net.Lumen.tech (4.69.219.218) icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 9 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 4.18.104.126 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 10 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
From 140.204.226.129 icmp_seq=1 Time to live exceeded

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 11 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ping www.certifiedhacker.com -t 12 -c 1
PING certifiedhacker.com (162.241.216.11) 56(84) bytes of data.
64 bytes from box5331.bluehost.com (162.241.216.11): icmp_seq=1 ttl=40 time=198 ms

--- certifiedhacker.com ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 198.123/198.123/198.123/0.000 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$
```

FIGURE 10 – TTL de 8 à 12 : la cible répond à TTL = 12

1.5.4 Analyse des résultats

TTL	Résultat	Correspondance
2	Time to live exceeded de 197.247.192.3	Saut 2
3	Time to live exceeded de 10.43.12.209	Saut 3
4	Time to live exceeded de 10.43.12.210	Saut 4
5	Time to live exceeded de 10.43.250.189	Saut 5
6, 7	Aucune réponse (100 % de perte)	Sauts 6 et 7 muets
8	Time to live exceeded de ae2.3601.edge2.Phoenix1.net.lumen.tech	Saut 8
9	Time to live exceeded de 4.18.104.126	Saut 9
10	Time to live exceeded de 140.204.226.129	Saut 10 (équilibre de charge)
11	Aucune réponse (100 % de perte)	Saut 11 muet
12	Réponse de la cible (box5331.bluehost.com, 198 ms)	Cible atteinte

TABLE 6 – Résultats du ping à TTL croissant

- **Sauts muets (TTL 6, 7 et 11)** : aucune réponse n'est reçue, ce qui correspond exactement aux * * * du *traceroute*. Ces routeurs détruisent bien le paquet, mais ne renvoient pas de message ICMP *Time Exceeded* (filtrage ou limitation de débit) ;
- **Saut 10** : l'adresse obtenue (140.204.226.129) diffère des trois vues dans le *traceroute* (140.91.194.165, 140.91.194.163, 140.204.226.83). Cela confirme la présence d'un **équilibre de charge** : plusieurs routeurs parallèles existent à ce niveau, et chaque sonde peut emprunter l'un d'eux ;
- **TTL reçu de la cible** : la réponse porte `ttl=40`, comme à la Tâche 1. Cette valeur décrit le chemin **retour** et ne peut pas être comparée directement au nombre de sauts aller.

Réponses aux questions de l'énoncé :

- À quelle valeur de TTL avez-vous reçu une réponse de la cible ?
→ À **TTL = 12**. Pour les valeurs inférieures, le paquet est détruit par un routeur intermédiaire (ou son signalement est filtré).
- Comment cette valeur se compare-t-elle au nombre de sauts trouvé avec *traceroute* ?
→ Elle est **identique** : le *traceroute* de la Tâche 4 comptait 12 sauts, et les adresses obtenues à chaque TTL correspondent saut par saut. Le TTL minimal pour atteindre la cible est donc égal au nombre de sauts du chemin aller.

1.6 Informations DNS avec nslookup

1.6.1 Principe

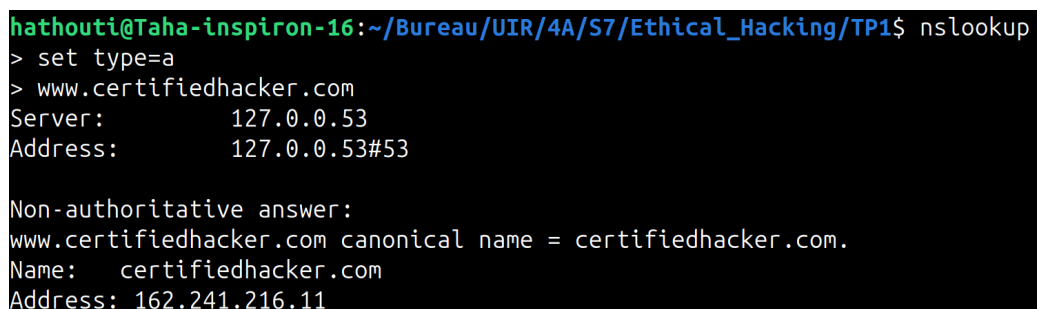
nslookup interroge le système DNS pour obtenir les enregistrements d'un domaine. Les types utilisés ici sont :

- **A** : associe un nom à une adresse IPv4;
- **CNAME** : alias d'un nom vers un autre nom (*canonical name*);
- **NS** : serveurs de noms faisant autorité pour la zone;
- **SOA** (*Start of Authority*) : paramètres administratifs de la zone, dont le serveur primaire.

Différence Windows / Linux : la syntaxe interactive est identique. Sous Ubuntu, la ligne **Server: 127.0.0.53** montre cependant que la requête passe par le **résolveur local systemd-resolved**, qui relaie et met en cache les réponses. Elles sont donc marquées *Non-authoritative answer*.

1.6.2 Requête de type A

```
1 nslookup
2 > set type=a
3 > www.certifiedhacker.com
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ nslookup
> set type=a
> www.certifiedhacker.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
www.certifiedhacker.com canonical name = certifiedhacker.com.
Name:   certifiedhacker.com
Address: 162.241.216.11
```

FIGURE 11 – Requête A sur `www.certifiedhacker.com`

La réponse indique que `www.certifiedhacker.com` est un **alias** (*canonical name*) de `certifiedhacker.com`, qui pointe vers `162.241.216.11`. Cette adresse est celle obtenue par ping à la Tâche 1 : les deux méthodes concordent.

1.6.3 Requêtes de type CNAME puis NS

```
1 > set type=cname
2 > certifiedhacker.com
3 > set type=ns
4 > certifiedhacker.com
```



```

> set type=cname
> certifiedhacker.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
*** Can't find certifiedhacker.com: No answer

Authoritative answers can be found from:
certifiedhacker.com
    origin = ns1.bluehost.com
    mail addr = dnsadmin.box5331.bluehost.com
    serial = 2026090818
    refresh = 86400
    retry = 7200
    expire = 3600000
    minimum = 300
> set type=ns
> certifiedhacker.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
certifiedhacker.com    nameserver = ns2.bluehost.com.
certifiedhacker.com    nameserver = ns1.bluehost.com.

Authoritative answers can be found from:
ns2.bluehost.com      internet address = 162.159.25.175
ns1.bluehost.com      internet address = 162.159.24.80
>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$

```

FIGURE 12 – Requêtes CNAME (sans réponse, SOA affiché) puis NS sur certifiedhacker.com

Requête CNAME : aucune réponse, mais un SOA. Le message `Can't find certifiedhacker.com: No answer` est normal : un domaine racine ne peut pas être un CNAME, puisqu'il porte déjà les enregistrements SOA et NS. En revanche, `nslookup` affiche l'enregistrement **SOA** de la zone, qui répond à la question de l'énoncé :

Champ	Valeur	Signification
origin	ns1.bluehost.com	Serveur de noms primaire de la zone
mail addr	dnsadmin.box5331.bluehost.com	Contact administratif (dnsadmin@box5331.bluehost.com)
serial	2026090818	Version de la zone, au format AAAAMMJnn : modifiée le 8 septembre 2026 (18 ^e révision)
refresh	86400 s (24 h)	Fréquence de vérification du primaire par les secondaires
retry	7200 s (2 h)	Délai avant nouvelle tentative en cas d'échec
expire	3600000 s ($\approx$ 41,7 j)	Durée après laquelle un secondaire cesse de répondre s'il ne joint plus le primaire
minimum	300 s (5 min)	Durée de mise en cache des réponses négatives

TABLE 7 – Décodage de l'enregistrement SOA de `certifiedhacker.com`

Requête NS. Deux serveurs de noms font autorité pour le domaine : `ns1.bluehost.com` (162.159.24.80) et `ns2.bluehost.com` (162.159.25.175). Le serveur `ns1` est bien celui désigné comme primaire par le SOA.

1.6.4 Intérêt en reconnaissance

- Les serveurs de noms confirment que le domaine est géré par **Bluehost**, cohérent avec le nom d'hôte `box5331.bluehost.com` de la Tâche 1 ;
- Le contact `dnsadmin@box5331.bluehost.com` et le nom `box5331` désignent le serveur physique d'hébergement ;
- Le numéro de série et les délais du SOA renseignent sur la fréquence de mise à jour de la zone ;
- Les serveurs primaire et secondaire sont les cibles d'un éventuel **transfert de zone** (AXFR), qui sera testé en Partie 2 avec `dig`.

Réponses aux questions de l'énoncé :

- *Quelle est l'adresse IP de `www.certifiedhacker.com` ?*
→ 162.241.216.11, obtenue via l'alias `certifiedhacker.com`.
- *Quel est le serveur de noms primaire du domaine cible ?*
→ `ns1.bluehost.com` (champ origin du SOA). Le serveur secondaire est `ns2.bluehost.com`.

1.7 Adresse IP du serveur de noms faisant autorité

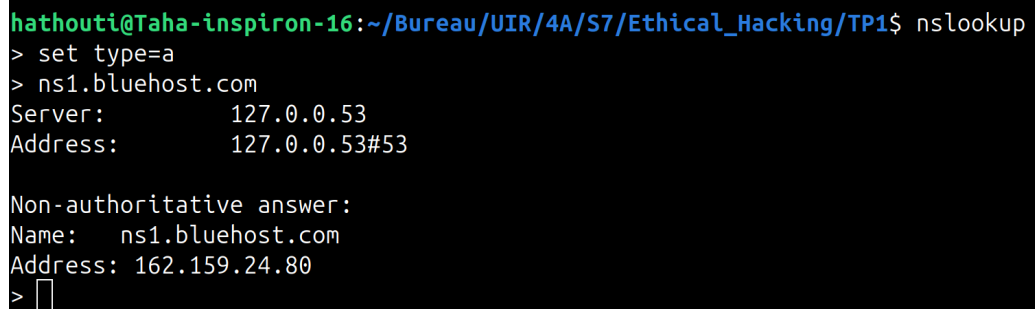
1.7.1 Principe

Un serveur de noms **faisant autorité** (*authoritative*) détient la version officielle des enregistrements d'une zone. Il s'oppose aux résolveurs récursifs (comme `systemd-resolved` sur Ubuntu) qui relaient les requêtes et mémorisent les réponses en cache. Cette tâche

consiste à trouver l'adresse IP du serveur primaire identifié à la Tâche 6, puis à l'interroger directement.

1.7.2 Requête de type A sur ns1.bluehost.com

```
1 nslookup
2 > set type=a
3 > ns1.bluehost.com
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ nslookup
> set type=a
> ns1.bluehost.com
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
Name:   ns1.bluehost.com
Address: 162.159.24.80
> █
```

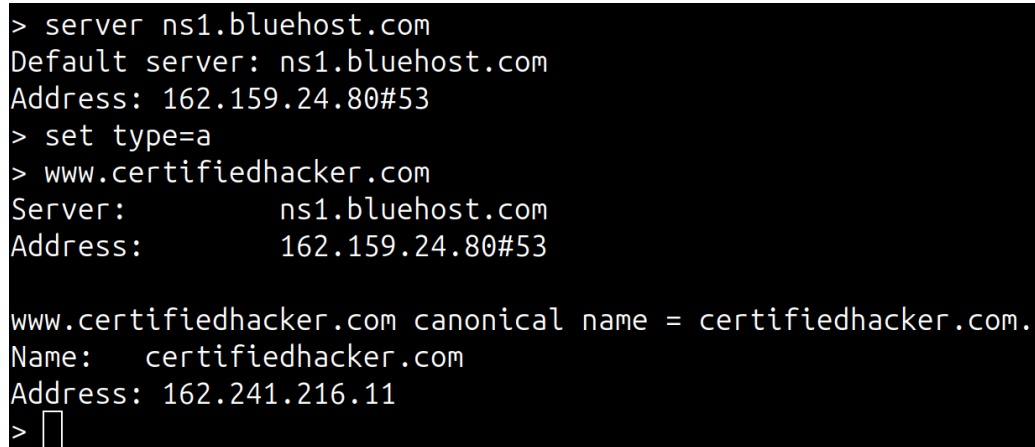
FIGURE 13 – Requête A sur ns1.bluehost.com via le résolveur local

L'adresse IP du serveur de noms primaire est 162.159.24.80, identique à celle déjà aperçue dans la section *Authoritative answers* de la Tâche 6. La réponse est marquée *Non-authoritative answer* : elle provient du résolveur local (127.0.0.53) et non du serveur officiel.

1.7.3 Interrogation directe du serveur faisant autorité

Pour illustrer la différence, nous redirigeons nslookup vers ns1.bluehost.com avec la commande **server**, puis interrogeons la cible :

```
1 > server ns1.bluehost.com
2 > set type=a
3 > www.certifiedhacker.com
```



```
> server ns1.bluehost.com
Default server: ns1.bluehost.com
Address: 162.159.24.80#53
> set type=a
> www.certifiedhacker.com
Server:          ns1.bluehost.com
Address:         162.159.24.80#53

www.certifiedhacker.com canonical name = certifiedhacker.com.
Name:   certifiedhacker.com
Address: 162.241.216.11
> █
```

FIGURE 14 – Interrogation directe de ns1.bluehost.com (162.159.24.80#53)

	Via le résolveur local	Via ns1.bluehost.com
Serveur interrogé	127.0.0.53	162.159.24.80
Mention <i>Non-authoritative</i>	Présente	Absente
Origine de la réponse	Cache / relais	Source officielle de la zone
Résultat	162.241.216.11	162.241.216.11

TABLE 8 – Comparaison réponse en cache et réponse faisant autorité

Les deux réponses sont identiques, mais seule la seconde est **garantie à jour**. Elle confirme aussi que `www.certifiedhacker.com` est un alias de `certifiedhacker.com`, pointant vers 162.241.216.11.

1.7.4 Intérêt en reconnaissance

Connaître l'IP du serveur faisant autorité permet de l'interroger directement, ce qui contourne le cache et évite de laisser des traces dans les journaux du résolveur. C'est aussi la cible naturelle d'un test de **transfert de zone** (AXFR), qui sera réalisé en Partie 2 avec `dig`.

Réponses aux questions de l'énoncé :

- *Quelle est l'adresse IP du serveur de noms primaire (ns1.bluehost.com) ?*
→ 162.159.24.80.
- *Pourquoi le serveur de noms faisant autorité est-il important pour la résolution DNS ?*
→ C'est la **source de vérité** de la zone : les résolveurs finissent toujours par l'interroger, directement ou non, pour obtenir un enregistrement qui n'est pas en cache. Les caches ne font que relayer ses réponses pendant la durée du TTL. Sans lui, un domaine ne serait plus résolvable une fois les caches expirés, et toute modification de zone (changement d'IP, de serveur de messagerie) doit y être publiée pour être prise en compte.

1.8 Footprinting via les moteurs de recherche (*Google Dorking*)

1.8.1 Principe

Le *Google Dorking* consiste à utiliser les **opérateurs de recherche avancés** de Google pour filtrer son index et faire remonter des pages ou des informations absentes d'une recherche classique. Il s'agit de **footprinting passif** : les requêtes sont adressées à Google et non aux serveurs de la cible, qui n'en garde aucune trace dans ses journaux. Ces requêtes se saisissent dans la barre de recherche d'un navigateur, et non dans un terminal.

Opérateur	Effet
site:	Limite la recherche à un domaine
filetype:	Ne retourne que les fichiers d'un format donné (PDF, DOCX, XLSX...)
inurl:	Ne retourne que les pages dont l'URL contient le terme
"..."	Recherche de la chaîne exacte

TABLE 9 – Opérateurs utilisés dans cette tâche

1.8.2 Recherche de documents PDF

```
1 site:certifiedhacker.com filetype:pdf
```

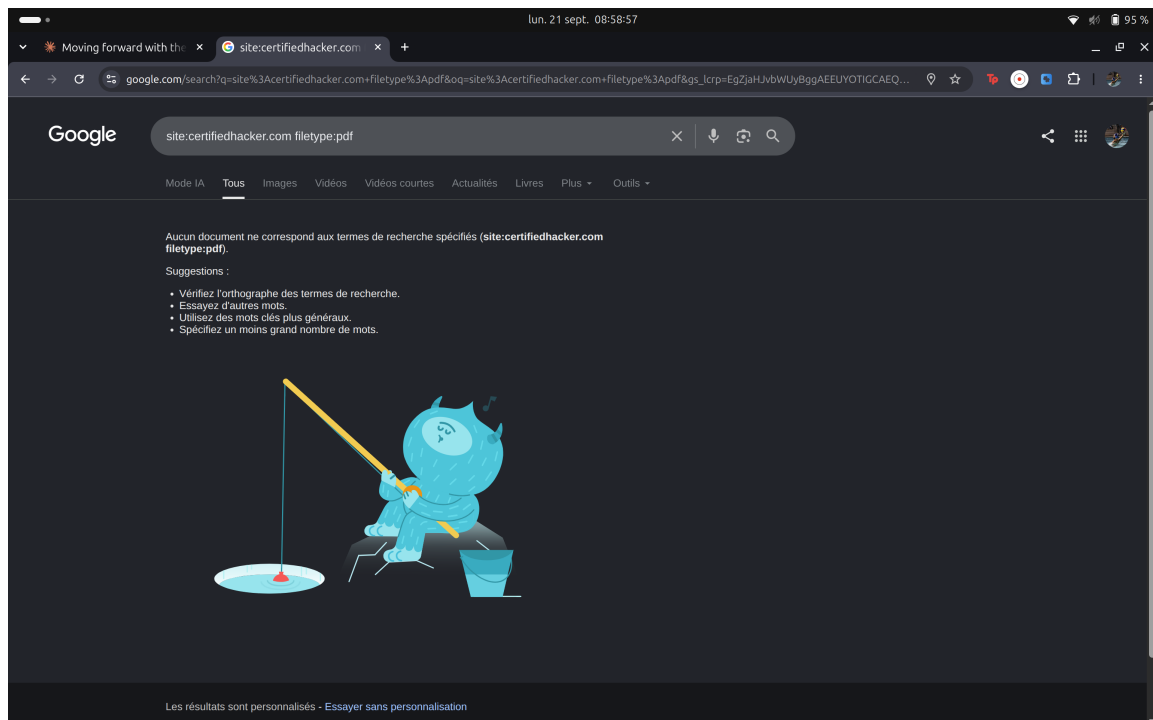


FIGURE 15 – Résultats de `site:certifiedhacker.com filetype:pdf`

Interprétation. Google répond « *Aucun document ne correspond aux termes de recherche spécifiés* » : **aucun PDF n'est indexé** sur le domaine. Ce résultat négatif est une information à part entière :

- le site **n'expose pas** de documents PDF au moteur de recherche, ce qui réduit l'intérêt de l'analyse des métadonnées (auteur, logiciel, chemins internes) qu'un attaquant mène habituellement sur ce type de fichiers ;
- l'absence de résultat **ne prouve pas l'absence de PDF** sur le serveur : un fichier peut exister sans être indexé (exclusion par `robots.txt`, absence de lien entrant, page trop récente). Elle indique seulement qu'il n'est pas accessible par ce canal.

En situation réelle, un attaquant essaierait d'autres formats sensibles (`docx`, `xlsx`, `txt`, `log`, `sql`), car les fuites concernent souvent des tableurs ou des sauvegardes plutôt que des PDF.

1.8.3 Recherche de portails de connexion

```
1 site:certifiedhacker.com inurl:login
```

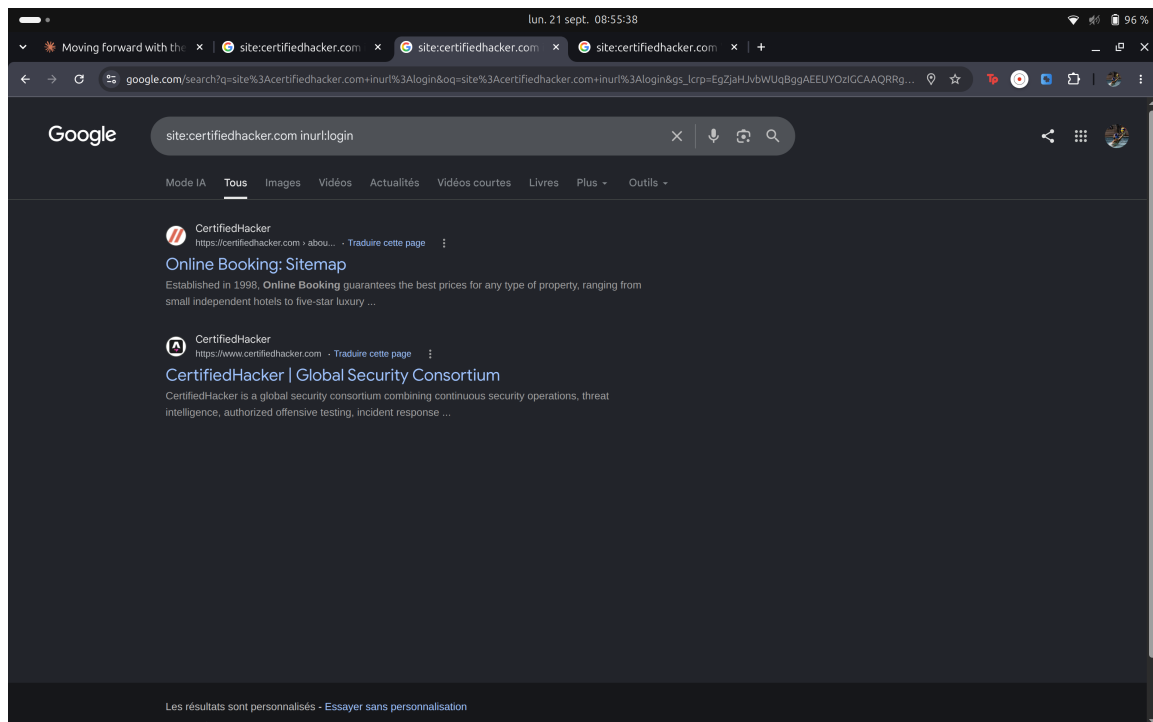


FIGURE 16 – Résultats de `site:certifiedhacker.com inurl:login`

Interprétation. La requête ne retourne que **deux résultats**, et aucun n'est un portail de connexion identifiable :

- *Online Booking : Sitemap* (`certifiedhacker.com`) : une page de plan de site d'un service de réservation ;
- la page d'accueil `www.certifiedhacker.com` (*Global Security Consortium*).

Le terme `login` n'apparaît pas dans les URL affichées (la première est tronquée) : la correspondance peut porter sur un chemin plus long ou sur le contenu de la page. L'absence de portail indexé est en soi une information : soit il n'existe pas, soit il est exclu de l'indexation. Le **plan de site** reste néanmoins intéressant pour un attaquant, car il liste l'arborescence du site et peut révéler des pages d'authentification ou d'administration non référencées ailleurs.

1.8.4 Recherche d'adresses e-mail

1 `site:certifiedhacker.com "@certifiedhacker.com"`

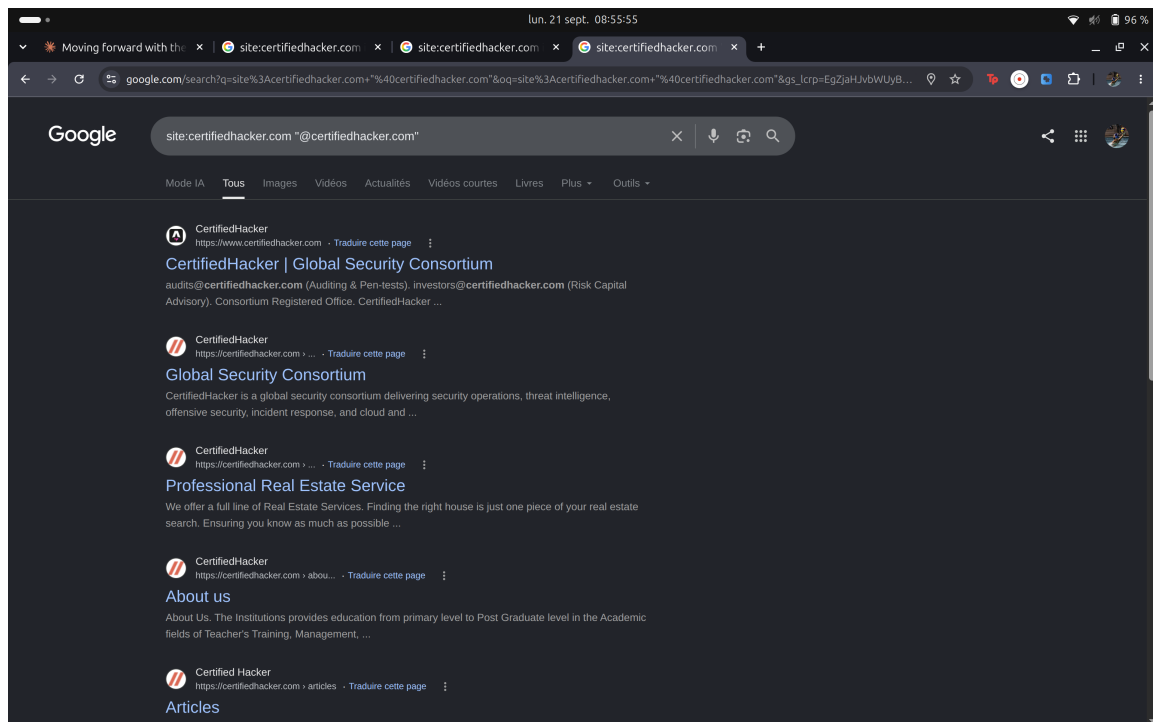


FIGURE 17 – Résultats de `site:certifiedhacker.com "@certifiedhacker.com"`

Interprétation. Cette requête est la plus productive. L'extrait du premier résultat (`www.certifiedhacker.com`) révèle **deux adresses e-mail** sans qu'il soit nécessaire d'ouvrir la page :

- `audits@certifiedhacker.com` : service d'audit et de tests d'intrusion (*Auditing & Pen-tests*);
- `investors@certifiedhacker.com` : service de conseil en capital-risque (*Risk Capital Advisory*).

Les autres résultats (*Professional Real Estate Service*, *About us*, *Articles*) portent sur l'immobilier, l'éducation et la gestion des ventes, sans rapport avec la sécurité : le domaine héberge des contenus hétérogènes, ce qui est cohérent avec un site d'entraînement. Les résultats proviennent à la fois de `www.certifiedhacker.com` et de `certifiedhacker.com`, ce qui rejoint le CNAME observé à la Tâche 6.

Exploitation par un attaquant :

- ce sont des adresses **fonctionnelles** (par service), qui révèlent l'organisation de l'entreprise et son activité;
- elles permettent de déduire la **convention de nommage** des adresses (`service@domaine`);
- elles servent de cibles pour des campagnes d'**hameçonnage** ou d'ingénierie sociale, par exemple en se faisant passer pour un investisseur auprès du service concerné.

1.8.5 Limites de la méthode

Les résultats de Google dépendent de la **date** (captures du 21 septembre 2026) et de la **personnalisation** (mention « Les résultats sont personnalisés » en bas de page). Ils ne sont donc pas strictement reproductibles d'un utilisateur à l'autre.

1.8.6 Synthèse des trois requêtes

Requête	Résultats	Information obtenue
<code>filetype:pdf</code>	0	Aucun document indexé
<code>inurl:login</code>	2	Plan de site d'un service de réservation, page d'accueil; aucun portail identifiable
<code>"@certifiedhacker.com"</code>	Plusieurs	Deux adresses e-mail fonctionnelles (<code>audits@</code> , <code>investors@</code>) et l'organisation des services

TABLE 10 – Comparaison de l'efficacité des trois dorks

Réponse à la question de l'énoncé :

- *Quel opérateur de recherche Google a été le plus efficace pour trouver des informations potentiellement sensibles ?*

→ La recherche de la chaîne exacte `"@certifiedhacker.com"` combinée à `site:.` Elle est la seule à avoir livré des données directement exploitables (deux adresses e-mail nominatives par service), alors que `filetype:pdf` n'a rien retourné et que `inurl:login` n'a pas révélé de portail d'authentification. Ce classement est propre à cette cible : sur un site réel, `filetype:` et `inurl:` sont souvent plus productifs (documents internes, pages d'administration exposées).

1.9 Footprinting via theHarvester

1.9.1 Principe

theHarvester automatise la collecte d'informations publiques (sous-domaines, adresses e-mail, IP, personnes) à partir de multiples sources : moteurs de recherche, journaux de certificats TLS, bases de renseignement, etc. C'est un outil de **footprinting passif** : aucune requête n'est envoyée directement à la cible.

1.9.2 Écart avec l'énoncé

L'énoncé propose `theharvester -d google.com -l 500 -b all` comme exemple, puis `-b google` sur la cible. La version installée (**theHarvester 5.0.0**) a retiré **Google** de ses sources, ce moteur bloquant trop systématiquement le scraping automatisé :

```
1 theHarvester -d certifiedhacker.com -l 300 -b google
```

The following engines are not supported: {'google'}

Nous avons donc utilisé deux sources ne nécessitant pas de clé API : **DuckDuckGo** (moteur de recherche) et **crt.sh** (journaux de transparence des certificats TLS).

1.9.3 Recherche via DuckDuckGo

```
1 theHarvester -d certifiedhacker.com -l 300 -b duckduckgo
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ theHarvester -d certifiedhacker.com -l 300 -b duckduckgo
*****
*
* theHarvester 5.0.0
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*
*****
[*] Activity: P0 passive collection

[*] Target: certifiedhacker.com

[*] Source workers: requested=3; effective=1.
[*] Searching Duckduckgo.

[*] No IPs found.

[*] No emails found.

[*] No people found.

[*] No hosts found.
```

FIGURE 18 – theHarvester avec la source duckduckgo : aucun résultat

Aucun résultat (IP, e-mail, personne, hôte) n'a été trouvé. DuckDuckGo indexe moins agressivement que Google et ne supporte pas l'opérateur `site:` de la même façon, ce qui limite fortement la découverte passive par ce biais sur un domaine de taille modeste.

1.9.4 Recherche via crt.sh (transparence des certificats)

```
1 theHarvester -d certifiedhacker.com -l 300 -b crtsh
```

```
mar. 22 sept. 13:53:56
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ theHarvester -d certifiedhacker.com -l 300 -b crtsh
*****
*
* theHarvester 5.0.0
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*
*****
[*] Activity: P0 passive collection

[*] Target: certifiedhacker.com

[*] Source workers: requested=3; effective=1.
[*] Searching Crtsh.

[*] No IPs found.

[*] No emails found.

[*] No people found.

[*] Hosts found: 53
-----
85368558.certifiedhacker.com
autoconfig.certifiedhacker.com
autodiscover.certifiedhacker.com
blog.certifiedhacker.com
caldren.certifiedhacker.com
ciphershield.certifiedhacker.com
cpanel.certifiedhacker.com
demo.certifiedhacker.com
events.certifiedhacker.com
fleet.certifiedhacker.com
iam.certifiedhacker.com
```

FIGURE 19 – theHarvester avec la source crtsh : 53 hôtes trouvés (extrait)

Principe de la source. Depuis 2018, tout certificat TLS public émis doit être journalisé dans un registre public (*Certificate Transparency*). Le nom de domaine et tous ses **noms alternatifs** (SAN) figurant sur un certificat y sont donc consultables, y compris pour des sous-domaines jamais liés depuis une page web ni indexés par un moteur de recherche.

Résultat : 53 sous-domaines découverts , classés par fonction :

Catégorie	Sous-domaines et interprétation
Sécurité opérationnelle	soc, ir, threatintelligence, offensive, iam, ciphershield, trustcenter — confirme le positionnement <i>Global Security Consortium</i> déjà observé dans les dorks Google (Tâche précédente)
Messagerie	mail, webmail, autoconfig, autodiscover, mail.demo — autoconfig/autodiscover sont des enregistrements standards de configuration automatique de client mail
Administration / accès distant	cpanel, webdisk, sftp, fleet — cpanel et webdisk confirment l'hébergement Bluehost identifié à la Tâche 1 (nom d'hôte inverse box5331.bluehost.com)
Communication	blog, news, events, notifications, demo
Téléphonie	pstn — suggère une passerelle voix (VoIP/PSTN)
Identifiants générés	85368558, website-215f0f34, website-25f58cec — motif numérique/hexadécimal typique d'un hébergement mutualisé multi-tenant
Noms de code	caldren, kestrava, tavrixon, itf — sans signification évidente, probablement des projets ou produits internes

TABLE 11 – Classification des 53 sous-domaines découverts via crt.sh

1.9.5 Intérêt en reconnaissance

- Les sous-domaines **cpanel**, **webdisk** et **sftp** sont des **surfaces d'attaque directes** : panneaux d'administration et accès aux fichiers, souvent ciblés par force brute si l'authentification est faible ;
- les sous-domaines liés à la sécurité (**soc**, **iam**, **trustcenter**) révèlent l'**organisation interne** et peuvent héberger des interfaces d'authentification sensibles ;

- les identifiants générés automatiquement confirment l'hébergement mutualisé, cohérent avec l'analyse de la Tâche 1 : d'autres sites peuvent être hébergés côte à côte sur la même infrastructure Bluehost ;
- chaque sous-domaine constitue une cible à **résoudre et scanner** en Partie 2 (**nmap**), certains pouvant héberger des services obsolètes ou mal configurés que le domaine principal ne présente pas.

Réponse à la question de l'énoncé :

- *Quelles informations issues des réseaux sociaux avez-vous trouvées qui pourraient servir à l'ingénierie sociale ?*
→ **theHarvester** n'a trouvé ni compte de réseau social ni personne nommée (aucune source ne l'a permis sur cette cible). En revanche, la liste des sous-domaines révèle l'**organigramme fonctionnel** de l'entreprise (SOC, IAM, IR, offensive, threat intelligence), ce qui permettrait à un attaquant de cibler ses e-mails d'hameçonnage sur les bons services ou de se faire passer pour un employé d'une équipe précise (par exemple, un faux message du SOC vers l'équipe IAM).

1.10 Analyse de dépôts OSINT avancés (Shodan et pagodo)

1.10.1 Principe

L'énoncé demande d'étudier deux dépôts GitHub représentatifs de l'automatisation de la reconnaissance passive. Ils prolongent les techniques manuelles vues plus haut (dorking, theHarvester) en les industrialisant.

jakejarvis/awesome-shodan-queries. Il s'agit d'une collection de requêtes prêtes à l'emploi pour **Shodan**, le « moteur de recherche des objets connectés ». Là où Google indexe des pages web, **Shodan scanne en permanence Internet et collecte les bannières de services** (ports ouverts, versions logicielles, certificats). Les requêtes du dépôt montrent comment localiser, à l'échelle mondiale : des **webcams** non protégées, des **bases de données** exposées sans authentification (MongoDB, Elasticsearch), des systèmes industriels **ICS/SCADA**, ou encore des serveurs faisant tourner une version logicielle vulnérable précise. C'est de la reconnaissance **entièrement passive du point de vue de la cible** : les données proviennent des scans de Shodan, pas d'un scan émis par l'attaquant.

opsdisk/pagodo. C'est un outil Python d'automatisation du *Google Dorking* (*Passive Google Dork*). Il fonctionne en deux temps : **ghdb_scraper.py** récupère la base de dorks de la **Google Hacking Database** (GHDB) de l'Exploit-DB, puis **pagodo.py** exécute ces dorks automatiquement contre un domaine cible. Il automatise donc, à grande échelle, exactement ce que nous avons fait à la main plus haut (**site:**, **filetype:**, **inurl:**) pour dénicher portails d'administration, fichiers de configuration et documents sensibles indexés par erreur.

1.10.2 Réponse à la question de l'énoncé (Shodan)

- *Quel type de données collecteriez-vous via Shodan, et comment un attaquant pourrait-il les exploiter ?*
→ Shodan fournit passivement : les **bannières de services** et **versions logicielles**

(Apache, nginx, OpenSSH, MySQL...), les **ports ouverts**, l’empreinte d’OS, la **géolocalisation** du serveur et les **métadonnées des certificats TLS**. Un attaquant s’en sert pour dresser une cartographie détaillée de la cible et repérer des services obsolètes ou mal configurés **sans jamais scanner directement le réseau** (donc sans laisser de trace dans les journaux de la cible). Sur `certifiedhacker.com`, Shodan révélerait vraisemblablement **les mêmes services que ceux que nous découvrirons activement au nmap en Partie 2** (MySQL 5.7.44 et PostgreSQL exposés, Exim 4.100...) — mais de façon totalement passive, ce qui illustre la puissance de l’OSINT.

Rappel : les deux autres questions de cette partie ont été traitées plus haut — l’opérateur le plus efficace (Google Dorking) et les informations exploitables pour l’ingénierie sociale (theHarvester).

2 Partie 2 : Footprinting via WHOIS, DNS, reconnaissance réseau et analyse de site web

Cette partie approfondit la reconnaissance : après la collecte passive de la Partie 1, elle interroge directement les bases d’enregistrement (WHOIS) et le DNS, puis passe à une reconnaissance **semi-active** (scan de ports, analyse du serveur web). Elle a été réalisée depuis la seconde machine (Arch Linux) sur le même réseau et la même cible ; on retrouve donc les constantes de la Partie 1 — IP 162.241.216.11, hébergement **Bluehost**, **12 sauts** de distance.

2.1 WHOIS Footprinting

2.1.1 Principe

Le protocole **WHOIS** interroge les bases des registres (*registries*) et des bureaux d’enregistrement (*registrars*) pour obtenir les informations administratives d’un nom de domaine : titulaire, dates de création et d’expiration, registrar, serveurs de noms, contacts. C’est une source de reconnaissance purement passive (aucune requête vers la cible elle-même).

```
1 whois certifiedhacker.com
```

```
[ygghest@ygghest-ExMach-Lap ~]$ whois certifiedhacker.com

Domain Name: CERTIFIEDHACKER.COM
Registry Domain ID: 88849376_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.networksolutions.com
Registrar URL: http://networksolutions.com
Updated Date: 2026-05-30T06:36:42Z
Creation Date: 2002-07-30T00:32:00Z
Registry Expiry Date: 2027-07-30T00:32:00Z
Registrar: Network Solutions, LLC
Registrar IANA ID: 2
Registrar Abuse Contact Email: domain.operations@web.com
Registrar Abuse Contact Phone: +1.8777228662
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: NS1.BLUEHOST.COM
Name Server: NS2.BLUEHOST.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-09-18T17:48:14Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
database through the use of electronic processes that are high-volume and
automated except as reasonably necessary to register domain names or
modify existing registrations; the Data in VeriSign Global Registry
Services' ("VeriSign") Whois database is provided by VeriSign for
information purposes only, and to assist persons in obtaining information
about or related to a domain name registration record. VeriSign does not
guarantee its accuracy. By submitting a Whois query, you agree to abide
by the following terms of use: You agree that you may use this Data only
for lawful purposes and that under no circumstances will you use this Data
to: (1) allow, enable, or otherwise support the transmission of mass
unsolicited, commercial advertising or solicitations via e-mail, telephone,
or facsimile; or (2) enable high volume, automated, electronic processes
that apply to VeriSign (or its computer systems). The compilation,
repackaging, dissemination or other use of this Data is expressly
prohibited without the prior written consent of VeriSign. You agree not to
use electronic processes that are automated and high-volume to access or
query the Whois database except as reasonably necessary to register
domain names or modify existing registrations. VeriSign reserves the right
to restrict your access to the Whois database in its sole discretion to ensure
operational stability. VeriSign may restrict or terminate your access to the
```

FIGURE 20 – Enregistrement WHOIS de certifiedhacker.com

2.1.2 Analyse des résultats

Champ	Valeur
Nom de domaine	CERTIFIEDHACKER.COM
Registry Domain ID	88849376_DOMAIN_COM-VRSN
Registrar	Network Solutions, LLC (IANA ID 2)
Serveur WHOIS du regis- trar	whois.networksolutions.com
Date de création	2002-07-30
Dernière mise à jour	2026-05-30
Date d'expiration	2027-07-30
Titulaire (<i>registrant</i>)	PERFECT PRIVACY, LLC (service de confidentialité)
Contact d'abus	domain.operations@web.com — +1.8777228662
Serveurs de noms	NS1.BLUEHOST.COM, NS2.BLUEHOST.COM
Statut du domaine	clientTransferProhibited
DNSSEC	unsigned (non signé)

TABLE 12 – Informations WHOIS de certifiedhacker.com

Points remarquables pour la reconnaissance :

- **Ancienneté** : le domaine est enregistré depuis **2002** (plus de 23 ans). C'est un domaine établi et crédible, et non un domaine jetable créé pour une campagne ponctuelle ;
- **Serveurs de noms Bluehost** : NS1/NS2.BLUEHOST.COM confirment l'hébergeur déjà identifié en Partie 1 (box5331.bluehost.com) ;
- **Titulaire masqué** : le WHOIS du registrar (whois.networksolutions.com) révèle que le titulaire (*registrant*) et le contact technique sont tous deux **PERFECT PRIVACY, LLC**, avec une adresse générique (5335 Gate Parkway, Jacksonville, FL), un téléphone +1.5707088622 et un e-mail anonymisé (kq9t994x73e@networksolutionspriv...). C'est un **service de confidentialité** qui remplace les vraies coordonnées du propriétaire : bonne pratique de protection de la vie privée, qui prive l'attaquant de noms et d'adresses directement exploitables ;
- **clientTransferProhibited** : un verrou anti-transfert est activé, protégeant contre le détournement de domaine (*domain hijacking*) — signe d'une hygiène de sécurité correcte ;
- **DNSSEC: unsigned** : la zone **n'est pas signée**. C'est une **faiblesse** : sans DNSSEC, les réponses DNS ne sont pas authentifiées cryptographiquement, ce qui laisse la porte à l'empoisonnement de cache et à l'usurpation de réponses DNS.

2.2 DNS Footprinting

2.2.1 Principe

dig et **nslookup** énumèrent les enregistrements DNS d'un domaine (A, NS, MX, SOA, etc.). Ils permettent de cartographier l'infrastructure : serveurs de noms, serveurs de messagerie, sous-domaines.

2.2.2 Énumération avec dig

```
1 dig certifiedhacker.com any
```

```

      .+*
     .o+*
    .ooo/
   +oooo:
  +ooooooo:
 +ooooooo+:
./:-!:+oooo+:
./+++/+++++++:
./+++++++:
./+++ooooooo+oooo/
./oooo5555+o555555+
.o555555+.../o55555+
-o555555.    :5555555.
:os555555/   os5555+!+.
./os555555/  +5555ooo-/
'os555555/!-  -:/+os5555+-
+55o+!-      .-./+o5o:
+!+.         .-!+/
+!+.         '

```

```

ygghest@ygghest-ExMach-Lap
-----
OS: Arch Linux x86_64
Host: ASUS TUF Gaming A15 FA506QM_TUF506QM (1.0)
Kernel: Linux 7.2.6-arch2-1
Uptime: 23 hours, 52 mins
Packages: 812 (pacman)
Shell: bash 5.3.15
Display (NCP004D): 1920x1080 in 15", 60 Hz [Built-in]
Window Manager: Hyprland 0.56.2 (Wayland)
Cursor: default (24px)
Terminal: kitty 0.48.2
Terminal Font: NotoSansMono-Regular (11pt)
CPU: AMD Ryzen 7 5800H (16) @ 4.46 GHz
GPU 1: NVIDIA GeForce RTX 3060 Mobile / Max-Q [Discrete]
GPU 2: AMD Radeon Vega Series / Radeon Vega Mobile Series [Integrated]
Memory: 9.34 GiB / 22.87 GiB (41%)
Swap: 0 B / 4.00 GiB (0%)
Disk (/): 216.06 GiB / 381.93 GiB (57%) - btrfs
Local IP (wlp3s0): 192.168.11.115/24
Battery (A32-K55): 100% [AC Connected]
Locale: en_US.UTF-8

```



```

[ygghest@ygghest-ExMach-Lap ~]$ dig certifiedhacker.com any

;<<>> DiG 9.20.29 <<>> certifiedhacker.com any
;; global options: +cmd
;; Got answer:
;; -->HEADER<- opcode: QUERY, status: NOERROR, id: 11287
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: 4096
; COOKIE: 312de76eb7e38160010000006aad7a07c7f624e1825c0853 (good)
;; QUESTION SECTION:
;certifiedhacker.com.                IN      ANY

;; ANSWER SECTION:
certifiedhacker.com.    86400   IN      NS      ns2.bluehost.com.
certifiedhacker.com.    86400   IN      NS      ns1.bluehost.com.
certifiedhacker.com.    3789    IN      HINFO    "RFC8482" ""

;; Query time: 129 msec
;; SERVER: 41.214.140.4#53(41.214.140.4) (TCP)
;; WHEN: Fri Sep 18 18:51:03 +01 2026
;; MSG SIZE rcvd: 145

[ygghest@ygghest-ExMach-Lap ~]$

```


- **Double SSH** : ports 22 et **2222** en OpenSSH 9.9 (version récente, correctement à jour). Le port 2222 est un accès SSH alternatif fréquent sur les hébergements mutualisés ;
- **Port 25 filtré** : cohérent avec le blocage sortant du SMTP standard, courant chez les FAI ;
- **Serveur DNS** : ISC BIND 9.16.23 confirme l'infrastructure DNS vue plus haut.

2.3.3 Détection du système d'exploitation

```
1 sudo nmap -O certifiedhacker.com
```

```

Shell: bash 5.3.15
Display (NCP004D): 1920x1080 in 15", 60 Hz [Built-in]
Window Manager: Hyprland 0.56.2 (Wayland)
Cursor: default (24px)
Terminal: kitty 0.48.2
Terminal Font: MonoSANSMono-Regular (11pt)
CPU: AMD Ryzen 7 5800H (16) @ 4.46 GHz
GPU 1: NVIDIA GeForce RTX 3060 Mobile / Max-Q [Discrete]
GPU 2: AMD Radeon Vega Series / Radeon Vega Mobile Series [Integrated]
Memory: 9.53 GiB / 22.87 GiB (42%)
Swap: 0 B / 4.00 GiB (0%)
Disk (/): 216.06 GiB / 381.93 GiB (57%) - btrfs
Local IP (wlp0s0): 192.168.11.115/24
Battery (BAT-005): 100% [AC Connected]
Locale: en_US.UTF-8

[ygghest@ggghest-ExMach-Lap ~]$ sudo nmap -O certifiedhacker.com
[sudo] password for ygghest:
Starting Nmap 7.90i (https://nmap.org) at 2026-09-18 18:58 +0100
Nmap scan report for certifiedhacker.com (162.241.216.11)
Host is up (0.25s latency).
OS record for 162.241.216.11: 11.216.241.162.in-addr.arpa
Not shown: 984 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
25/tcp    filtered smtp
26/tcp    open  rsftp
53/tcp    open  domain
80/tcp    open  http
110/tcp   open  pop3
143/tcp   open  imap
443/tcp   open  https
465/tcp   open  smtps
987/tcp   open  submission
993/tcp   open  imap
995/tcp   open  pop3s
2222/tcp  open  EtherNetIP-1
3306/tcp  open  mysql
5432/tcp  open  postgresql
Aggressive OS guesses: Vyatta router (Linux 2.6.26) (97%), Linux 2.6.18 (97%), Netgear ReadyNAS Duo NAS device (RAIDiator 4.1.4) (97%), Linux 2.6.28 (96%), Zhone 6211-13 series ADSL2+ modem (94%), Linux 2.6.26 (93%), Linux 2.6.32 (93%), Linux 3.16 - 3.16 (93%), Canon ImageRUNNER ADVANCE C5051 printer (92%), Cisco Unified Communications Manager VoIP adapter (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 12 hops
OS detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 51.51 seconds
[ygghest@ggghest-ExMach-Lap ~]$

```

FIGURE 24 – `nmap -O` : tentative d'identification de l'OS et distance réseau

Analyse.

- **Aucune correspondance exacte d'OS** : les suppositions sont incohérentes (routeur Vyatta, NAS Netgear ReadyNAS, Linux 2.6.x). Cette **absence de correspondance est elle-même informative** : elle est typique d'une **infrastructure mutualisée et virtualisée** placée derrière des répartiteurs de charge, qui brouillent l'empreinte TCP/IP. Cela confirme l'hébergement partagé Bluehost. Le **Service Info** du scan `-sV` indiquait néanmoins un noyau **Linux** ;
- **Noms de service différents** : sans `-sV`, `nmap` affiche le nom de service associé au **numéro de port** dans `/etc/services` (d'où `26/rsftp`, `2222/EtherNetIP-1`), alors que `-sV` sonde réellement le service (d'où `26/Exim`, `2222/OpenSSH`). Ce contraste illustre la différence entre supposition et détection active ;
- **Distance réseau : 12 sauts** — **exactement** le nombre de sauts du **traceroute** de la Partie 1, alors qu'il s'agit d'une autre machine. La cohérence inter-parties est confirmée.

2.4 Analyse du site web (wget & Nikto)

2.4.1 Miroir du site avec wget

```
1 wget -r -l 1 certifiedhacker.com
```



```
[ygghest@ygghest-ExMach-Lap ~]$ wget -r -l 1 certifiedhacker.com
Prepended http:// to 'certifiedhacker.com'
--2026-09-18 19:07:09-- http://certifiedhacker.com/
Resolving certifiedhacker.com (certifiedhacker.com)... 162.241.216.11
Connecting to certifiedhacker.com (certifiedhacker.com)|162.241.216.11|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8500 (8.3K) [text/html]
Saving to: 'certifiedhacker.com/index.html'

certifiedhacker.com/index.html      100%[=====]

2026-09-18 19:07:09 (12.0 MB/s) - 'certifiedhacker.com/index.html' saved [8500/8500]

Loading robots.txt; please ignore errors.
--2026-09-18 19:07:09-- http://certifiedhacker.com/robots.txt
Reusing existing connection to certifiedhacker.com:80.
HTTP request sent, awaiting response... 200 OK
Length: 8500 (8.3K) [text/html]
Saving to: 'certifiedhacker.com/robots.txt'

certifiedhacker.com/robots.txt      100%[=====]

2026-09-18 19:07:10 (110 MB/s) - 'certifiedhacker.com/robots.txt' saved [8500/8500]

--2026-09-18 19:07:10-- http://certifiedhacker.com/assets/index-Ab5jICQq.js
Reusing existing connection to certifiedhacker.com:80.
HTTP request sent, awaiting response... 200 OK
Length: 405134 (396K) [text/javascript]
Saving to: 'certifiedhacker.com/assets/index-Ab5jICQq.js'

certifiedhacker.com/assets/index-Ab5jICQq.js  100%[=====]

2026-09-18 19:07:11 (423 KB/s) - 'certifiedhacker.com/assets/index-Ab5jICQq.js' saved [405134/405134]

--2026-09-18 19:07:11-- http://certifiedhacker.com/assets/index-C40ey7jD.css
Reusing existing connection to certifiedhacker.com:80.
HTTP request sent, awaiting response... 200 OK
Length: 43719 (43K) [text/css]
Saving to: 'certifiedhacker.com/assets/index-C40ey7jD.css'

certifiedhacker.com/assets/index-C40ey7jD.css  100%[=====]

2026-09-18 19:07:11 (109 KB/s) - 'certifiedhacker.com/assets/index-C40ey7jD.css' saved [43719/43719]

FINISHED --2026-09-18 19:07:11--
Total wall clock time: 2.4s
Downloaded: 4 files, 455K in 1.3s (343 KB/s)
[ygghest@ygghest-ExMach-Lap ~]$
```

FIGURE 25 – `wget -r -l 1` : miroir local du site (profondeur 1)

Analyse. Quatre fichiers sont récupérés (455 Ko au total) :

- `index.html` (8,5 Ko) et `robots.txt` ;
- `assets/index-Ab5jICQq.js` (396 Ko) et `assets/index-C40ey7jD.css` (43 Ko).

Les noms de fichiers **hachés** (`index-Ab5jICQq.js`) sont la signature d'un **build moderne de type Vite/Webpack** : le site est une **application monopage** (SPA, React/Vue). Le fichier `robots.txt` présent mérite d'être inspecté, car ses règles `Disallow` révèlent parfois des répertoires sensibles que l'administrateur souhaite cacher des moteurs.

2.4.2 Scan de vulnérabilités avec Nikto

```
1 nikto -h certifiedhacker.com
```

```
yoghest@ygghest-ExMach-Lap ~$ cat /etc/os-release
PRETTY_NAME="Arch Linux"
NAME=Arch Linux
ID=arch
BUILD_ID=rolling
VERSION=
VARIANT=""
VARIANT_IDENT=""
LOGO=arch.png
OS= Arch Linux x86_64
Host= ASUS TUF Gaming A15 FAS06QM_TUF580QM (1.0)
Kernel= Linux 7.2.6-arch2-1
Uptime= 1 day, 12 mins
Packages= 816 (pacman)
Shell= bash 5.3.15
Display= (MCP084D): 1920x1080 in 15", 60 Hz [Built-in]
Window Manager= Hyprland 0.56.2 (Wayland)
Cursor= default (2dpx)
Terminal= kitty 0.46.2
Terminal Font= NotoSansMono-Regular (11pt)
CPU= AMD Ryzen 7 5800H (16) @ 4.46 GHz
GPU 1= NVIDIA GeForce RTX 3080 Mobile / Max-Q [Discrete]
GPU 2= AMD Radeon Vega Series / Radeon Vega Mobile Series [Integrated]
Memory= 9.64 GiB / 22.87 GiB (42%)
Swap= 0 B / 4.00 GiB (0%)
Disk (/): 216.12 GiB / 381.93 GiB (57%) - btrfs
Local Tr (wlp3s0)= 192.168.11.115/24
Battery (A02-455)= 100% [AC Connected]
Locale= en_US.UTF-8
```



```
[ygghest@ygghest-ExMach-Lap ~]$ nikto -h certifiedhacker.com
- Nikto v2.6.0

-----
+ Target IP:      162.241.216.11
+ Target Hostname: certifiedhacker.com
+ Target Port:    80
+ Platform:       Unknown
+ Start Time:     2026-09-18 19:09:16 (GMT1)

-----
+ Server: nginx/1.29.8
+ ERROR: Failed to check for updates: 403
+ [999100] /: Uncommon header(s) 'x-proxy-cache' found, with contents: HIT.
+ [999100] /: Uncommon header(s) 'x-server-cache' found, with contents: true.
+ [999100] /: Uncommon header(s) 'host-header' found, with contents: c2zhcnwKLnJsdWVob3N0LmVubGVzQe=.
+ [999902] /: Server banner changed from "nginx/1.29.8" to "Apache".
+ [01357] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [01357] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [01357] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [01357] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [01357] /: Suggested security header missing: referer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referer-Policy
+ [999998] OPTIONS: Allowed HTTP Methods: GET, POST, OPTIONS, HEAD .
+ [999967] /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
```

FIGURE 26 – nikto -h certifiedhacker.com : en-têtes et configuration du serveur web

Analyse.

- **Architecture à deux couches** : le serveur s'annonce d'abord `nginx/1.29.8`, puis Nikto note que la bannière **change en Apache**. Il y a donc un **reverse proxy nginx** devant un **backend Apache** — information d'architecture précieuse ;
- **Couche de cache** : les en-têtes `x-proxy-cache: HIT` et `x-server-cache: true` révèlent un cache en frontal ;
- **Fuite d'infrastructure** : l'en-tête `host-header` en base64 se décode en une chaîne commençant par « `shared` », confirmant le **backend mutualisé Bluehost** ;
- **Méthodes HTTP** : `GET`, `POST`, `OPTIONS`, `HEAD` sont autorisées (pas de `PUT/DELETE` dangereux).

Vulnérabilités identifiées (en-têtes de sécurité manquants) et remédiations :

En-tête manquant	Risque et remédiation
Content-Security-Policy	Risque d'injection de scripts (XSS). Définir une politique CSP restrictive.
Strict-Transport-Security	Risque de <i>downgrade</i> HTTPS → HTTP et de MITM. Ajouter HSTS avec un max-age élevé.
X-Content-Type-Options	Risque de <i>MIME sniffing</i> . Ajouter nosniff .
Referrer-Policy	Fuite d'URL via l'en-tête Referer . Définir une politique restrictive.
Permissions-Policy	Contrôle insuffisant des API du navigateur. Restreindre les fonctionnalités autorisées.

TABLE 14 – En-têtes de sécurité absents et mesures correctives

2.5 Réponses aux questions de l'énoncé

- *Quelles informations utiles le WHOIS a-t-il fournies, et comment pourraient-elles être exploitées ?*
→ Le registrar (Network Solutions), les dates (création 2002, expiration 2027), les serveurs de noms Bluehost et le contact d'abus. Le titulaire étant masqué par un service de confidentialité, la principale exploitation reste le **spear-phishing** via le contact d'abus, et le repérage de l'échéance d'expiration (une expiration proche non renouvelée est une fenêtre de détournement). L'absence de **DNSSEC** ouvre par ailleurs la voie à l'usurpation de réponses DNS.
- *Qu'a révélé le footprinting DNS sur l'infrastructure ?*
→ Une infrastructure **Bluehost** (ns1/ns2, serveur ISC BIND 9.16.23 sous Red-Hat), protégée contre l'énumération ANY par la **RFC 8482**, mais **sans DNSSEC**. Les deux serveurs de noms constituent les cibles d'un éventuel transfert de zone.
- *Quelles vulnérabilités Nikto a-t-il identifiées, et comment les corriger ?*
→ Essentiellement l'**absence de plusieurs en-têtes de sécurité HTTP** (CSP, HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy). La correction consiste à **ajouter ces en-têtes** côté serveur (voir tableau 14), à forcer HTTPS et à durcir la configuration du serveur web.

2.6 Exercices théoriques (Task 2)

1. **Exercice 1 — Collecte d'informations DNS de example.com.** Déterminer l'adresse IP, les enregistrements MX et NS :

```
1 dig example.com A +short
2 dig example.com MX +short
3 nslookup -type=ns example.com
```

2. **Exercice 2 — Google Dorking de PDF sur sampledomain.com.**

```
1 site:sampledomain.com filetype:pdf cybersecurity
```

3. **Exercice 3 — Découverte de répertoires cachés sur `http://testwebsite.com`.** Avec un outil de brute-force de répertoires comme `gobuster` :

```
1 gobuster dir -u http://testwebsite.com -w /chemin/vers/  
   wordlist.txt
```

4. **Exercice 4 — Récolte d'e-mails sur `somedomain.com`.** L'outil de Kali Linux adapté est `theHarvester` :

```
1 theHarvester -d somedomain.com -b all
```

5. **Exercice 5 — Analyse d'un serveur Apache `http://webdomain.com`.** Scan de vulnérabilités avec `nikto` :

```
1 nikto -h http://webdomain.com
```

3 Partie 3 : Automatisation du footprinting (script shell)

3.1 Objectif

Développer un script Bash qui enchaîne automatiquement les principales tâches de reconnaissance (analyse DNS, découverte de sous-domaines, scan de ports, scan web) et organise les résultats dans un répertoire nommé d'après la cible.

3.2 Script

Listing 1 – Script d'automatisation du footprinting

```
1 #!/bin/bash  
2  
3 # Verifie qu'un domaine cible est fourni  
4 if [ -z "$1" ]; then  
5     echo "Usage: _$0_ <domaine-cible>"  
6     exit 1  
7 fi  
8  
9 DOMAIN="$1"  
10 OUTPUT_DIR="_$DOMAIN"  
11  
12 echo " [* ] _Creation_du_repertoire_pour_la_cible:_$DOMAIN"  
13 mkdir -p "$OUTPUT_DIR"  
14  
15 echo " [* ] _Analyse_DNS_avec_dnsenum..."  
16 dnsenum --nocolor "$DOMAIN" > "$OUTPUT_DIR/dnsenum_result.txt"  
17 2>&1  
18  
19 echo " [* ] _Recherche_de_sous-domaines_avec_Sublist3r..."  
20 sublist3r -d "$DOMAIN" -o "$OUTPUT_DIR/sublist3r_result.txt"  
21  
22 echo " [* ] _Scan_rapide_avec_Nmap..."
```

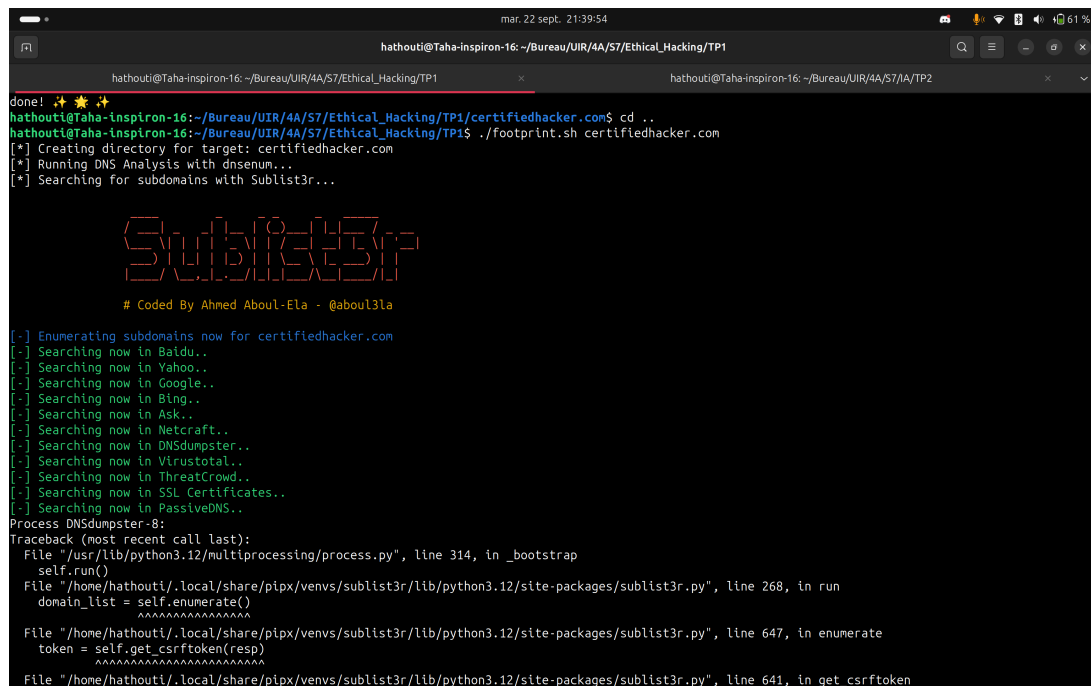
```

22 nmap -T4 -F "$DOMAIN" > "$OUTPUT_DIR/nmap_result.txt" 2>&1
23
24 echo " [*] Scan du serveur web avec Nikto..."
25 nikto -h "$DOMAIN" > "$OUTPUT_DIR/nikto_result.txt" 2>&1
26
27 echo " [*] Requete WHOIS..."
28 whois "$DOMAIN" > "$OUTPUT_DIR/whois_result.txt" 2>&1
29
30 echo " [+] Footprinting termine avec succes!"
31 echo " [+] Tous les resultats sont dans : ./ $OUTPUT_DIR/"

```

3.3 Exécution et observations

Après installation des outils manquants (`sudo apt install dnsenum nikto; pipx install sublist3r`), le script a été exécuté avec `./footprint.sh certifiedhacker.com`. Il crée le répertoire `certifiedhacker.com/` et y écrit un fichier par outil.



```

done! 🌟🌟🌟
hathouti@Taha-inspiron-16: ~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1/certifiedhacker.com$ cd ..
hathouti@Taha-inspiron-16: ~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$ ./footprint.sh certifiedhacker.com
[*] Creating directory for target: certifiedhacker.com
[*] Running DNS Analysis with dnsenum...
[*] Searching for subdomains with Sublist3r...

  SUBLIST3R

# Coded By Ahmed Aboul-Ela - @aboul3la

[.] Enumerating subdomains now for certifiedhacker.com
[.] Searching now in Baidu..
[.] Searching now in Yahoo..
[.] Searching now in Google..
[.] Searching now in Bing..
[.] Searching now in Ask..
[.] Searching now in Netcraft..
[.] Searching now in DNSdumpster..
[.] Searching now in Virustotal..
[.] Searching now in ThreatCrowd..
[.] Searching now in SSL Certificates..
[.] Searching now in PassiveDNS..
Process DNSdumpster-8:
Traceback (most recent call last):
  File "/usr/lib/python3.12/multiprocessing/process.py", line 314, in _bootstrap
    self.run()
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 268, in run
    domain_list = self.enumerate()
                  ~~~~~~
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 647, in enumerate
    token = self.get_csrf_token(resp)
            ~~~~~~
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 641, in get_csrf_token

```

FIGURE 27 – Lancement du script : création du répertoire, dnsenum puis Sublist3r

```

mar. 22 sept. 21:40:45
hathouti@Taha-inspiron-16: ~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1
hathouti@Taha-inspiron-16: ~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1
hathouti@Taha-inspiron-16: ~/Bureau/UIR/4A/S7/IA/TP2

[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSdumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
Process DNSdumpster-8:
Traceback (most recent call last):
  File "/usr/lib/python3.12/multiprocessing/process.py", line 314, in _bootstrap
    self.run()
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 268, in run
    domain_list = self.enumerate()
                  ^^^^^^^^^^^^^^^^^
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 647, in enumerate
    token = self.get_csrf_token(resp)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/hathouti/.local/share/pipx/venvs/sublist3r/lib/python3.12/site-packages/sublist3r.py", line 641, in get_csrf_token
    token = csrf_regex.findall(resp)[0]
           ~~~~~~^~~~~~
IndexError: list index out of range
[-] Error: Virustotal probably now is blocking our requests
[-] Saving results to file: certifiedhacker.com/sublist3r_result.txt
[-] Total Unique Subdomains Found: 5
www.certifiedhacker.com
autodiscover.certifiedhacker.com
cpanel.certifiedhacker.com
demo.certifiedhacker.com
mail.certifiedhacker.com
[*] Running quick Nmap scan...
[*] Running Nikto web server scan...
[*] Running Whois lookup (Additional tool)...
[*] Footprinting completed successfully! All results saved in directory: ./certifiedhacker.com/
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP1$

```

FIGURE 28 – Fin d’exécution : sous-domaines de Sublist3r, puis Nmap, Nikto, WHOIS

Plusieurs enseignements se dégagent des fichiers de résultats :

- **dnsenum** — **transfert de zone refusé** : les deux serveurs de noms répondent AXFR record query failed: NOTIMP au test de transfert de zone. C’est un **résultat de sécurité positif pour la cible** : l’AXFR est désactivé, ce qui empêche de récupérer l’intégralité de la zone DNS d’un coup. L’outil confirme aussi l’enregistrement **MX** (mail.certifiedhacker.com → 162.241.216.11) : le serveur de messagerie partage la même IP que le web, cohérent avec l’hébergement mutualisé. Le *brute-force* de sous-domaines (dictionnaire dns.txt) trouve autodiscover, blog, ftp, mail, pop, smtp, webmail, www. Enfin, la *reverse-DNS* sur toute la plage 162.241.216.0/24 donne **0 résultat sur 256** adresses (PTR non exposés) ;
- **Sublist3r** — **sources partiellement cassées** : l’outil interroge une dizaine de sources ; DNSdumpster lève une exception (IndexError, jeton CSRF introuvable car le site a changé) et Virustotal bloque les requêtes. C’est le signe d’un outil **vieillissant** dont certaines sources ne sont plus à jour. Il agrège malgré tout **5 sous-domaines** : www, autodiscover, cpanel, demo, mail ;
- **Complémentarité des techniques de découverte de sous-domaines** : les trois méthodes employées dans ce TP ne trouvent pas les mêmes résultats, ce qui plaide pour les combiner :

Méthode	Nombre	Nature
crt.sh (Partie 1)	53	Transparence des certificats (le plus exhaustif)
dnsenum (brute-force)	8	Dictionnaire de noms courants
Sublist3r (moteurs)	5	Scraping de moteurs de recherche (bridé)

TABLE 15 – Sous-domaines découverts selon la technique

Chaque source apporte des noms que les autres manquent (`cpanel/demo` n'apparaissent que via `Sublist3r`; `blog/ftp/pop/smtp/webmail` uniquement via `dnsenum`). La **transparence des certificats** (`crt.sh`) reste de loin la plus complète.

- **Scan rapide vs scan complet** : le `nmap -F` (100 ports courants) s'exécute en ≈ 2 s et retrouve l'essentiel des services de la Partie 2, mais **manque le port 2222** (SSH alternatif, hors des 100 ports par défaut) que le scan complet `-sV` avait détecté. C'est le compromis classique **vitesse contre exhaustivité**;
- **Nikto** — **version trop ancienne** : la version empaquetée sous Ubuntu (**v2.1.5**) renvoie `0 host(s) tested`, sans effectuer le scan. L'analyse Nikto exploitable est donc celle de la **Partie 2** (Nikto **v2.6.0**, en-têtes de sécurité manquants). Pour un usage automatisé, il faudrait installer une version récente de Nikto;
- **WHOIS** : la sortie confirme les données de la Partie 2, y compris le titulaire masqué *PERFECT PRIVACY, LLC* obtenu via le WHOIS du registrar.

3.4 Réponses aux questions de l'énoncé

1. **Quelle commande pour extraire les informations DNS de la cible ?**

L'outil `dnsenum` (option `--nocolor` pour un fichier texte propre) :

```
1 dnsenum --nocolor "<domaine-cible>"
```

2. **Comment sauvegarder les sous-domaines dans un fichier ?**

`Sublist3r` avec l'option de sortie `-o` :

```
1 sublist3r -d "<domaine-cible>" -o "sublist3r_result.txt"
```

3. **Quelle est la commande Nmap pour un scan rapide ?**

Le drapeau `-F` (*fast scan*, 100 ports courants) avec un timing agressif `-T4` :

```
1 nmap -T4 -F "<domaine-cible>"
```

4. **Comment rediriger la sortie d'une commande vers un fichier ?**

Avec les opérateurs de redirection `>` (écrase) ou `>>` (ajoute); `2>&1` redirige aussi la sortie d'erreur :

```
1 commande > "fichier.txt" 2>&1
```

5. **Comment vérifier si un répertoire existe, et le créer sinon ?**

Le plus simple est `mkdir -p`, qui crée le répertoire uniquement s'il n'existe pas déjà (sans erreur sinon) :

```
1 mkdir -p "<nom-repertoire>"
```

Ou de façon explicite avec le test `-d` :

```
1 if [ ! -d "$OUTPUT_DIR" ]; then
2     mkdir "$OUTPUT_DIR"
3 fi
```